

SQL Practice — Student Marks Table

ASSIGNMENT

Name: Hansika

Roll No: 2520030237

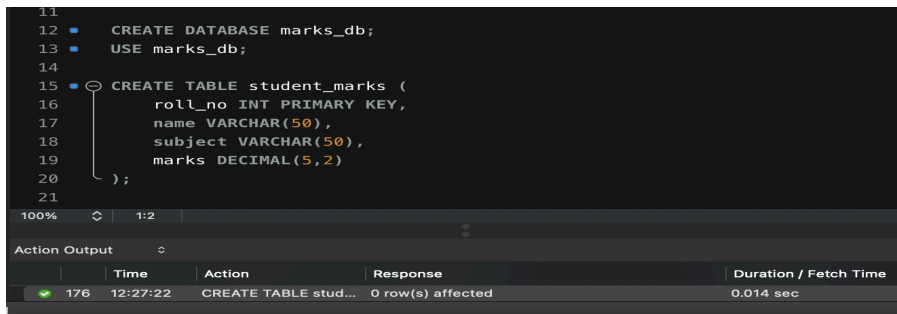
Section: 7

Email: gunapatihansikareddy@klh.edu.in

Step 1 — Create Database and Table

```
CREATE DATABASE marks_db;
USE marks_db;

CREATE TABLE student_marks (
    roll_no INT PRIMARY KEY,
    name VARCHAR(50),
    subject VARCHAR(50),
    marks DECIMAL(5,2)
);
```



```
11
12 CREATE DATABASE marks_db;
13 USE marks_db;
14
15 CREATE TABLE student_marks (
16     roll_no INT PRIMARY KEY,
17     name VARCHAR(50),
18     subject VARCHAR(50),
19     marks DECIMAL(5,2)
20 );
21
```

	Time	Action	Response	Duration / Fetch Time
176	12:27:22	CREATE TABLE stud...	0 row(s) affected	0.014 sec

Step 2 — Insert Data

```
INSERT INTO student_marks (roll_no, name, subject, marks) VALUES
(1, 'Hansika', 'Math', 85.50),
(2, 'Sita', 'Math', 92.75),
(3, 'Anil', 'Math', 78.40),
(4, 'Priya', 'Math', 88.90),
(5, 'Vijay', 'Math', 80.25),
(6, 'Subbu', 'AWS', 98.50),
(7, 'Dwaraka', 'DBMS', 95.75),
(8, 'Ranjani', 'English', 97.40),
(9, 'Kavitha', 'AWS', 99.90),
(10, 'Seetha', 'Azure', 82.25);
```

```
22 • INSERT INTO student_marks (roll_no, name, subject, marks) VALUES
23     (1, 'HANSIKA', 'Math', 85.50),
24     (2, 'Sita', 'Math', 92.75),
25     (3, 'Anil', 'Math', 78.40),
26     (4, 'Priya', 'Math', 88.90),
27     (5, 'Vijay', 'Math', 80.25),
28     (6, 'Subbu', 'AWS', 98.50),
29     (7, 'Dwaraka', 'DBMS', 95.75),
30     (8, 'Ranjani', 'English', 97.40),
31     (9, 'Kavitha', 'AWS', 99.90),
32     (10, 'Seetha', 'Azure', 82.25);
33
34
```

100% 25:18

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 177	12:28:35	INSERT INTO studen...	10 row(s) affected Records: 10 Duplicates: 0...	0.0093 sec

Step 3 — Simple Select

```
SELECT * FROM student_marks;
```

Object I... Session
No object selected

```
33
34 • SELECT * FROM student_marks;
35
```

100% 31:29

Result Grid Filter Rows: Search Edit: Export/Import:

	roll_no	name	subject	marks	
	1	HANSIKA	Math	85.50	
	2	Sita	Math	92.75	
	3	Anil	Math	78.40	
	4	Priya	Math	88.90	
	5	Vijay	Math	80.25	
	6	Subbu	AWS	98.50	
	7	Dwaraka	DBMS	95.75	
	8	Ranjani	English	97.40	
	9	Kavitha	AWS	99.90	
	10	Seetha	Azure	82.25	
	NULL	NULL	NULL	NULL	

student_marks 1 App

Step 4 — Aggregate Functions

```
SELECT COUNT(*) AS total_students FROM student_marks;
SELECT SUM(marks) FROM student_marks;
SELECT AVG(marks) FROM student_marks;
SELECT MAX(marks) FROM student_marks;
SELECT MIN(marks) FROM student_marks;
```

LINE BY LINE OUTPUT —>

```
COUNT(*) -> 10
SUM(marks) -> 899.55
AVG(marks) -> 89.955
MAX(marks) -> 99.90
MIN(marks) -> 78.40
```

The screenshot shows a SQL IDE with a dark theme. The SQL editor contains five queries, numbered 37 to 41, each followed by a blue execution icon. The queries are: 37. SELECT COUNT(*) AS total_students FROM student_marks; 38. SELECT SUM(marks) FROM student_marks; 39. SELECT AVG(marks) FROM student_marks; 40. SELECT MAX(marks) FROM student_marks; 41. SELECT MIN(marks) FROM student_marks;. Below the editor, a toolbar shows '100%' zoom, a refresh icon, '1:36' execution time, and buttons for 'Result Grid', 'Filter Rows', 'Search', and 'Export'. The 'Result Grid' is active, displaying a table with one column 'MIN...' and one row containing the value '78.40'.

```
37 • SELECT COUNT(*) AS total_students FROM student_marks;
38 • SELECT SUM(marks) FROM student_marks;
39 • SELECT AVG(marks) FROM student_marks;
40 • SELECT MAX(marks) FROM student_marks;
41 • SELECT MIN(marks) FROM student_marks;
42
```

100% 1:36

Result Grid Filter Rows: Search Export:

MIN...
78.40

Step 5 — Filtering with WHERE

```
SELECT * FROM student_marks WHERE marks > 85;
SELECT * FROM student_marks WHERE marks >= 90;
SELECT * FROM student_marks WHERE marks < 80;
SELECT * FROM student_marks WHERE marks BETWEEN 80 AND 90;
SELECT * FROM student_marks WHERE name LIKE 'P%';
SELECT * FROM student_marks WHERE name IN ('Ravi', 'Sita', 'Vijay');
SELECT * FROM student_marks WHERE marks > 85 AND (subject = 'Math' OR name LIKE 'P%');
```

LINE BY LINE OUTPUT —>

```
marks > 85: Sita, Priya, Subbu, Dwaraka, Ranjani, Kavitha
marks >= 90: Sita, Subbu, Dwaraka, Ranjani, Kavitha
marks < 80: Anil
BETWEEN 80 AND 90: Hansika, Priya, Vijay, Seetha
name LIKE 'P%': Priya
name IN (...): Hansika, Sita, Vijay
marks>85 AND (...): Sita, Priya
```

```

45 • SELECT * FROM student_marks WHERE marks > 85;
46 • SELECT * FROM student_marks WHERE marks >= 90;
47 • SELECT * FROM student_marks WHERE marks < 80;
48 • SELECT * FROM student_marks WHERE marks BETWEEN 80 AND 90;
49 • SELECT * FROM student_marks WHERE name LIKE 'P%';
50 • SELECT * FROM student_marks WHERE name IN ('Ravi', 'Sita', 'Vijay');
51 • SELECT * FROM student_marks WHERE marks > 85 AND (subject = 'Math' OR name LIKE 'P%');
52

```

100% 46:47

Result Grid Filter Rows: Search Edit: Export/Import:

roll_no	name	subject	marks
1	HANSIKA	Math	85.50
2	Sita	Math	92.75
4	Priya	Math	88.90
NULL	NULL	NULL	NULL

Result Grid Form Editor

Step 6 — Update Queries

```

UPDATE student_marks SET marks = 90.00 WHERE roll_no = 3;
UPDATE student_marks SET subject = 'Remedial Math' WHERE marks < 80;

SELECT * FROM student_marks;

```

```

57 • SELECT * FROM student_marks;
58

```

100% 1:52 1 error found

Result Grid Filter Rows: Search Edit: Export/Import:

roll_no	name	subject	marks
1	HANSIKA	Math	85.50
2	Sita	Math	92.75
3	Anil	Math	90.00
4	Priya	Math	88.90
5	Vijay	Math	80.25
6	Subbu	AWS	98.50
7	Dwaraka	DBMS	95.75
8	Ranjani	English	97.40
9	Kavitha	AWS	99.90
10	Seetha	Azure	82.25
NULL	NULL	NULL	NULL

Result Grid Form Editor Field Types

student_marks 23 Apply Revert

Step 7 — Delete Queries

```

DELETE FROM student_marks WHERE roll_no = 5;
DELETE FROM student_marks WHERE marks < 75;

SELECT * FROM student_marks;

```

```

59 • DELETE FROM student_marks WHERE roll_no = 5;
60 • DELETE FROM student_marks WHERE marks < 75;
61
62 • SELECT * FROM student_marks;
63

```

100% 1:58 1 error found

Result Grid Filter Rows: Search Edit: Export/Import:

	roll_no	name	subject	marks
	1	HANSIKA	Math	85.50
	2	Sita	Math	92.75
	3	Anil	Math	90.00
	4	Priya	Math	88.90
	6	Subbu	AWS	98.50
	7	Dwaraka	DBMS	95.75
	8	Ranjani	English	97.40
	9	Kavitha	AWS	99.90
	10	Seetha	Azure	82.25
	NULL	NULL	NULL	NULL

Step 8 — Sorting

```

SELECT * FROM student_marks ORDER BY marks ASC;
SELECT * FROM student_marks ORDER BY marks DESC;
SELECT * FROM student_marks ORDER BY name ASC;
SELECT * FROM student_marks ORDER BY name DESC;

```

LINE BY LINE OUTPUT —>

marks ASC: Seetha, Hansika, Priya, Anil, Sita, Dwaraka, Ranjani, Subbu, Kavitha
marks DESC: reverse of above (Kavitha first, Seetha last)
name ASC / DESC: alphabetical order of names

```

68 • SELECT * FROM student_marks ORDER BY marks ASC;
69 • SELECT * FROM student_marks ORDER BY marks DESC;
70 • SELECT * FROM student_marks ORDER BY name ASC;
71 • SELECT * FROM student_marks ORDER BY name DESC;
72

```

100% 48:71 1 error found

Result Grid Filter Rows: Search Edit: Export/Import:

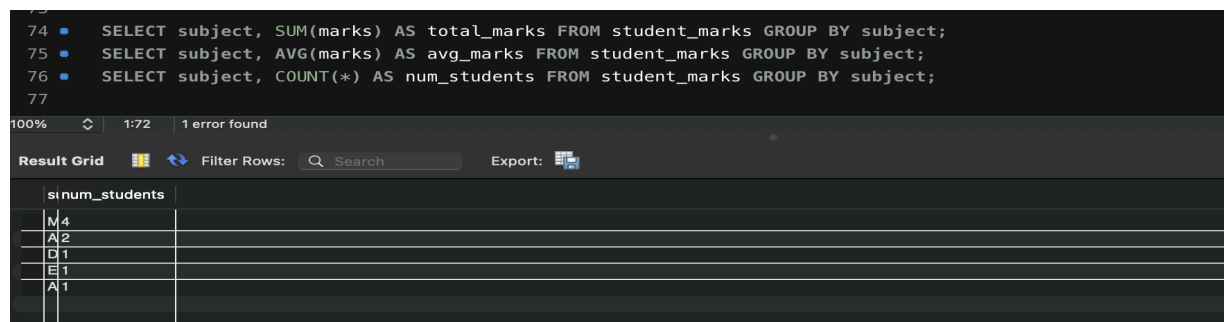
	roll_no	name	subject	marks
	6	Subbu	AWS	98.50
	2	Sita	Math	92.75
	10	Seetha	Azure	82.25
	8	Ranjani	English	97.40
	4	Priya	Math	88.90
	9	Kavitha	AWS	99.90
	1	HANSIKA	Math	85.50
	7	Dwaraka	DBMS	95.75
	3	Anil	Math	90.00
	NULL	NULL	NULL	NULL

Step 9 — Group By

```
SELECT subject, SUM(marks) AS total_marks FROM student_marks GROUP BY subject;
SELECT subject, AVG(marks) AS avg_marks FROM student_marks GROUP BY subject;
SELECT subject, COUNT(*) AS num_students FROM student_marks GROUP BY subject;
```

LINE BY LINE OUTPUT —>

Math: SUM 357.15, AVG 89.29, COUNT 4
AWS: SUM 198.40, AVG 99.20, COUNT 2
DBMS: SUM 95.75, AVG 95.75, COUNT 1
English: SUM 97.40, AVG 97.40, COUNT 1
Azure: SUM 82.25, AVG 82.25, COUNT 1



The screenshot shows a SQL IDE with three queries entered in the editor:

```
74 • SELECT subject, SUM(marks) AS total_marks FROM student_marks GROUP BY subject;
75 • SELECT subject, AVG(marks) AS avg_marks FROM student_marks GROUP BY subject;
76 • SELECT subject, COUNT(*) AS num_students FROM student_marks GROUP BY subject;
77
```

Below the editor, the 'Result Grid' shows the output of the third query:

subject	num_students
M	4
A	2
D	1
E	1
A	1

Step 10 — Having

```
SELECT subject, AVG(marks) AS avg_marks
FROM student_marks
GROUP BY subject
HAVING AVG(marks) > 90;
```

```
SELECT subject, COUNT(*) AS num_students
FROM student_marks
GROUP BY subject
HAVING COUNT(*) > 1;
```

LINE BY LINE OUTPUT —>

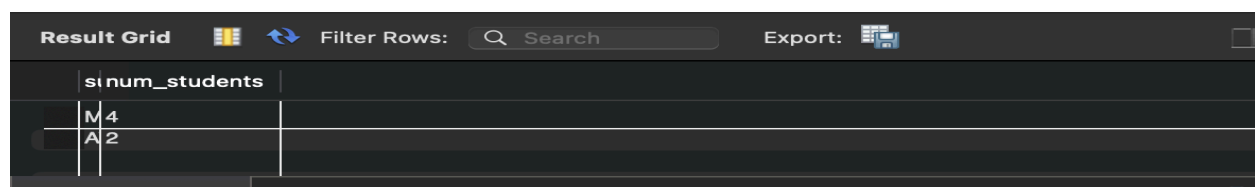
AVG > 90: AWS (99.20), DBMS (95.75), English (97.40)
COUNT > 1: Math (4), AWS (2)



The screenshot shows a SQL IDE with a query result displayed in a grid:

subject	avg_marks
A	99.20
D	95.75
E	97.40

The grid is labeled 'Result 3' and has a 'Read Only' status.



The screenshot shows a SQL IDE with a query result displayed in a grid:

subject	num_students
M	4
A	2

The grid is labeled 'Result 4' and has a 'Read Only' status.